

# Object-Oriented Programming (OOP) — C++

## Հարցազրույցի / քննության ամբողջական ուղեցույց

Այս presentation-ը ընդգրկում է OOP-ի բոլոր հիմնական թեմաները C++ կոդով՝ 4 սյուները (Encapsulation, Inheritance, Polymorphism, Abstraction), inheritance-ի տեսակները, access modifier-ները, abstract class vs interface, object-ների միջև հարաբերությունները (association/aggregation/composition), SOLID սկզբունքները, և պատրաստի interview պատասխանները:

## 0. Ի՞նչ է OOP-ը

OOP-ը (Object-Oriented Programming) ծրագրավորման մոտեցում է, որտեղ ծրագիրը կառուցվում է **Object-ների (օբյեկտների)** շուրջ, ոչ թե միայն ֆունկցիաների ու տրամաբանության:

Class -> blueprint / template (նախագիծ)  
Object -> class-ի instance (իրական օբիեկտ հիշողության մեջ)

```
class Car {  
public:  
    std::string brand;  
};  
  
Car car1; // car1-ը Car class-ի object է
```

Անալոգիա. **Class** -ը տան նախագիծն է (գծագիր), **Object** -ը՝ այդ նախագծով կառուցված իրական տունը: Մեկ նախագծից կարելի է շատ տուն կառուցել:

## 1. Encapsulation (Կապսուլացում)

Տվյալների և մեթոդների միավորումն է մեկ class-ի մեջ, և տվյալների ուղիղ հասանելիության սահմանափակումը: Դաշտերը հայտարարում ենք **private**, հասանելիություն տալիս **getter/setter** -ներով:

```
class Person {
private:
    std::string name;
public:
    std::string getName() const { return name; }
    void setName(const std::string& n) { name = n; }
};
```

## Ինչու է կարևոր — 4 առավելություն

**Data Hiding.** Առանց encapsulation-ի ոչ ոք չի պաշտպանում տվյալները.

```
class BankAccount { public: double balance; };
acc.balance = -100000; // սխալ արժեք, ոչ ոք չի արգելում
```

Encapsulation-ով վերահսկում ես.

```
class BankAccount {
private:
    double balance = 0;
public:
    void setBalance(double b) {
        if (b >= 0) balance = b; // validation
    }
};
```

**Security.** `private std::string password;` նշանակում է, որ արտաքին կողմ չի կարող ուղիղ դիպչել password-ին:

**Better Control.** Դու ես որոշում, թե ինչպես են տվյալները փոփոխվում.

```
void setAge(int age) {
    if (age > 0 && age < 120) this->age = age; // setAge(-50) չի ընդունվի
}
```

**Easy Maintenance.** Եթե ներքին ներկայացումը փոխես, բայց `getName()/setName()` -ը մնա նույնը, արտաքին կողմ չի կոտրվի:

## Interview պատասխան

*Encapsulation is the process of wrapping data and methods into a single unit (class) and restricting direct access to data using access modifiers.*

## 2. Inheritance (Ժառանգում)

Մեկ class-ը ստանում է մյուսի հատկություններն ու մեթոդները:

```
class Animal {  
public:  
    void eat() { std::cout << "Eating"; }  
};  
  
class Dog : public Animal { }; // Dog-ը կարող է օգտագործել eat()
```

### Առավելություններ

**Code Reusability + Reduced Duplication.** Ընդհանուր կոդը գրում ես մեկ անգամ base class-ում.

```
class Car { public: void start() {} };  
  
class BMW : public Car {};  
class Audi : public Car {};  
class Mercedes : public Car {};
```

// բոլորը reuse են անում start()

**Easier Maintenance.** `Car::start()` -ը փոխելիս բոլոր child class-երը ավտոմատ ստանում են փոփոխությունը:

### Inheritance-ի տեսակները

- |                 |                         |                         |
|-----------------|-------------------------|-------------------------|
| 1. Single       | A -> B                  | (մեկ parent, մեկ child) |
| 2. Multilevel   | A -> B -> C             | (շղթա)                  |
| 3. Hierarchical | A -> B, A -> C          | (մեկ parent, շատ child) |
| 4. Multiple     | A, B -> C               | (շատ parent, մեկ child) |
| 5. Hybrid       | վերևիների համադրություն |                         |

C++-ը, ի տարբերություն Java-ի, **multiple inheritance class-ներով թույլատրում է** (`class C : public A, public B`), և Diamond Problem-ը լուծվում է `virtual` inheritance-ով.

```
class A {};  
class B {};  
class C : public A, public B {}; // multiple inheritance -- OK C++-ում
```

### 3. Diamond Problem

Multiple inheritance-ի հիմնական խնդիրը.



Եթե `A`-ն ունի `show()`, և `B`, `C` երկուսն էլ ժառանգում են, ապա `D`-ն կունենա `A`-ի երկու պատճեն -> երկիմաստություն (ambiguity):

```
class A { public: void show() { std::cout << "A"; } };  
class B : public A {};  
class C : public A {};  
class D : public B, public C {};  
  
D d;  
// d.show(); // ERROR: ambiguous -- A::show()-ի երկու պատճեն (B-ից ու C-ից)
```

### C++-ի լուծումը — virtual inheritance

```
class A { public: void show() { std::cout << "A"; } };  
class B : virtual public A {}; // virtual -> A-ն մեկ պատճեն  
class C : virtual public A {};  
class D : public B, public C {};  
  
D d;  
d.show(); // OK -- A-ն մեկ անգամ է (virtual inheritance)
```

`virtual` inheritance-ը երաշխավորում է, որ base class-ը ( `A` ) ունենա **մեկ** ընդհանուր պատճեն `D`-ում:

---

## 4. Polymorphism (Բազմաձևություն)

---

Poly (many) + Morph (forms). մեկ interface, տարբեր վարք:

### Compile-time Polymorphism — Function Overloading

Նույն անուն, տարբեր parameters. compiler-ը ընտրում է ըստ argument-ների:

```
class MathUtil {
public:
    int    add(int a, int b)           { return a + b; }
    int    add(int a, int b, int c)    { return a + b + c; }
    double add(double a, double b)    { return a + b; }
};
```

### Runtime Polymorphism — Function Overriding (virtual)

Child class-ը վերասահմանում է parent-ի `virtual` մեթոդը. ընտրությունը runtime-ում է ըստ object-ի իրական տիպի:

```
class Animal {
public:
    virtual void sound() { std::cout << "Animal sound"; }
    virtual ~Animal() {}
};

class Dog : public Animal {
public:
    void sound() override { std::cout << "Bark"; }
};

Animal* a = new Dog();
a->sound(); // "Bark" -> runtime-ում որոշվում է
delete a;
```

Կարևոր. C++-ում overriding-ը պահանջում է `virtual` base class-ում (Java-ում բոլոր non-static մեթոդները default-ով virtual են):

## Overloading vs Overriding

|             | Overloading       | Overriding      |
|-------------|-------------------|-----------------|
| Ժամանակ     | Compile-time      | Runtime         |
| Signature   | տարբեր parameters | նույն signature |
| virtual     | պետք չէ           | պահանջվում է    |
| Inheritance | պետք չէ           | պահանջվում է    |

## Interview պատասխան

*Polymorphism allows objects to take many forms. One interface can be used for different implementations.*

## 5. Abstraction (Աբստրակցիա)

Ցույց ենք տալիս միայն անհրաժեշտ տեղեկատվությունը, թաքցնում ներքին implementation-ը: C++-ում abstraction-ը արվում է **abstract class**-ով (pure virtual function, `= 0`):

```
class Animal {
public:
    virtual void sound() = 0;    // pure virtual -> abstract class
    virtual ~Animal() {}
};

// Animal a;    // ERROR: abstract class-ից object չի ստեղծվում

class Dog : public Animal {
public:
    void sound() override { std::cout << "Bark"; }
};
```

Անալոգիա. `car.start()` անելիս չգիտես, թե ներսում վառելիքի պոմպը ինչպես աշխատեց: Քեզ միայն `start()` -ն է հետաքրքրում: Դա abstraction-ն է:

## Առավելություններ

Security -> օգտվողը իմանում է ինչ, ոչ ինչպես (թաքցված implementation)  
Less Complexity -> պարզ interface բարդ ներսի փոխարեն  
Better Design -> ընդհանուր contract, մաքուր կառուցվածք  
Easy Maintenance-> նոր implementation ավելացնելը չի կոտրում մնացածը

Օրինակ՝ payment համակարգ.

```
class Payment {
public:
    virtual void pay(double amount) = 0;
    virtual ~Payment() {}
};

class CardPayment : public Payment { public: void pay(double a) override {} };
class PayPalPayment : public Payment { public: void pay(double a) override {} };
class CryptoPayment : public Payment { public: void pay(double a) override {} }; // նոր, մնացածը
```

## Interview պատասխան

*Abstraction is hiding implementation details and showing only essential information.*

## 6. Abstract Class vs Interface (C++)

C++-ում առանձին `interface` keyword չկա. interface-ը սարքվում է **pure abstract class**-ով (միայն pure virtual մեթոդներ, առանց state):

```
// Abstract class (state + implementation + pure virtual)
class Animal {
protected:
    std::string name;           // state
public:
    void eat() { std::cout << "Eating"; } // implementation ունի
    virtual void sound() = 0;           // pure virtual
    virtual ~Animal() {}
};

// Interface (միայն pure virtual, առանց state)
class Drawable {
public:
    virtual void draw() = 0;           // միայն contract
    virtual ~Drawable() {}
};
```

|                      | Abstract Class         | Interface (pure abstract)  |
|----------------------|------------------------|----------------------------|
| Implementation       | կարող է ունենալ        | սովորաբար ոչ               |
| State (fields)       | կարող է ունենալ        | ոչ                         |
| Constructor          | ունի                   | ոչ                         |
| Multiple inheritance | հնարավոր (virtual)     | ապահով (state չկա)         |
| Կիրառ                | «is-a» + ընդհանուր կող | մաքուր capability/contract |

Կանոն. abstract class՝ երբ ընդհանուր կող/state կա; interface (pure abstract)՝ մաքուր contract, multiple inheritance-ի համար:

## 7. Access Modifiers (C++)

| Modifier  | Նույն class | Derived class | Դրսից |
|-----------|-------------|---------------|-------|
| private   | այո         | ոչ            | ոչ    |
| protected | այո         | այո           | ոչ    |
| public    | այո         | այո           | այո   |



```

class Parent {
private:    int a = 1;    // child չի տեսնում
protected: int b = 2;    // child տեսնում է
public:    int c = 3;    // բոլորը տեսնում են
};

class Child : public Parent {
public:
    void show() {
        // std::cout << a;    // ERROR (private)
        std::cout << b << c; // OK (protected, public)
    }
};

```

## Inheritance-ի access (C++-ին հատուկ)

C++-ում ժառանգման ձևն ինքը ունի access.

```

class D1 : public Base    {};    // public    -> public մնում public
class D2 : protected Base {};    // protected-> public դառնում protected
class D3 : private Base  {};    // private    -> ամեն ինչ դառնում private

```

Հիմնականում օգտագործում են `public` inheritance (իրական «is-a»):

## Private-ը ժառանգվում է

Տեխնիկապես `private` դաշտը object-ի հիշողության մեջ կա, բայց child class-ը ուղիղ չի կարող օգտագործել: Հասանելի է միայն `public` / `protected` getter-ով.

```

class Parent {
private:
    int age = 10;
public:
    int getAge() const { return age; }
};

class Child : public Parent {
public:
    void print() { std::cout << getAge(); }    // OK getter-ով
};

```

## 8. Object-ների միջև հարաբերությունները

Inheritance-ը «is-a» է (Dog **is an** Animal): Բայց կա նաև «has-a» (composition/aggregation):

### Association (կապ)

Երկու անկախ object իրար հետ կապված են (օր. Teacher և Student):

### Aggregation (has-a, թույլ)

Ամբողջը պարունակում է մասը, բայց մասը կարող է ապրել առանց ամբողջի.

```
class Department {  
    std::vector<Teacher*> teachers;    // Teacher-ը կարող է գոյություն ունենալ առանց Department-ի  
};
```

### Composition (has-a, ուժեղ)

Մասը չի կարող ապրել առանց ամբողջի (ամբողջը ոչնչանալիս մասն էլ).

```
class Engine {  
public:  
    void start() { std::cout << "Engine started"; }  
};  
  
class Car {  
private:  
    Engine engine;    // Car HAS-A Engine (Car ոչնչանալիս Engine-ն էլ)  
public:  
    void startCar() { engine.start(); }  
};
```

Կանոն. **Composition over Inheritance** — հաճախ ավելի ճկուն է object-ներ կցելը (has-a), քան խորը inheritance շղթա կառուցելը (is-a):

## 9. SOLID սկզբունքները

Mid-level interview-ի համար կարևոր: Լավ OOP դիզայնի 5 սկզբունք.

S - Single Responsibility -> ամեն class մեկ պատասխանատվություն  
O - Open/Closed -> բաց ընդլայնման, փակ փոփոխման  
L - Liskov Substitution -> child-ը պետք է փոխարինի parent-ին առանց կոտրելու  
I - Interface Segregation -> շատ փոքր interface > մեկ մեծ  
D - Dependency Inversion -> կախված եղիր abstraction-ից, ոչ concrete class-ից

**Single Responsibility.** `Invoice` class-ը չպետք է և հաշվարկի, և տպի, և PDF սարքի. բաժանիր:

**Open/Closed.** Նոր payment տեսակ ավելացնելիս նոր class ես ավելացնում ( `: public Payment` ), հինը չես փոխում:

**Liskov.** Եթե `Bird` -ը ունի `fly()` , ապա `Penguin : public Bird` -ը կոտրում է Liskov-ը (penguin-ը չի թռչում) -> դիզայնը սխալ է:

**Interface Segregation.** Մի ստիպիր class-ին implement անել մեթոդներ, որ պետք չեն. բաժանիր interface-ը փոքրերի:

**Dependency Inversion.** Բարձր level կողմ կախված լինի `Payment` abstract class-ից, ոչ `CardPayment` concrete class-ից:

## 10. Interview Short Answers

What is OOP?

*A programming paradigm based on objects and classes.*

4 pillars of OOP?

*Encapsulation, Inheritance, Polymorphism, Abstraction.*

Encapsulation?

*Wrapping data and methods into a single unit and restricting direct access to data.*

Inheritance?

*Mechanism by which one class acquires properties and behaviors of another class.*

Polymorphism?

*Ability of an object to take multiple forms; one interface, many implementations.*

## **Abstraction?**

*Hiding implementation details and showing only essential information.*

## **Abstract class vs Interface (C++)?**

*Abstract class can have implementation and state; a pure abstract class (only pure virtual, no state) acts as an interface.*

## **Overloading vs Overriding?**

*Overloading = same name, different parameters, compile-time; Overriding = same signature, runtime, needs virtual + inheritance.*

## **Why virtual destructor?**

*To ensure the derived destructor runs when deleting through a base pointer (avoids leaks).*

## **Composition vs Inheritance?**

*Inheritance = "is-a"; Composition = "has-a". Prefer composition for flexibility.*

---

# Ամփոփում (Cheat Sheet)

---

4 ՍՅՈՒՆ (մեկ նախադասությամբ):

Encapsulation -> Hide Data (private + getter/setter)  
Inheritance -> Reuse Code (: public Base / is-a)  
Polymorphism -> Many Forms (overload compile / virtual override runtime)  
Abstraction -> Hide Implementation (pure virtual = 0 / abstract class)

INHERITANCE TYPES:

Single, Multilevel, Hierarchical, Multiple, Hybrid

ACCESS (նեղից լայն):

private < protected < public  
inheritance access: class D : public/protected/private Base

ABSTRACT CLASS vs INTERFACE (C++):

abstract -> implementation + state  
interface -> pure abstract (միայն = 0, առանց state)

RELATIONSHIPS:

is-a -> Inheritance  
has-a -> Aggregation (թույլ) / Composition (ուժեղ)

SOLID: Single-Responsibility, Open/Closed, Liskov, Interface-Segregation, Dependency-Inversion

DIAMOND PROBLEM -> C++: virtual inheritance

VIRTUAL DESTRUCTOR -> polymorphic base-ի համար ՊԱՐՏԱԴԻՐ